ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Type parametrization and subtyping

We have seen various examples of parametrization at work.

Now, it is time for a deeper dive and, more importantly, to study how type parametrization *interacts* with subtyping.

The combination raises questions that have no obvious answer: if `String <: Any`, should `Queue[String] <: Queue[Any]` follow? The answer depends on what the queue does with its elements, and the discipline that captures the answer — *variance* — is the main subject of the lecture.

We start by introducing a parametric type that we will use as the running example throughout: a functional `Queue`.

Queue[A]

As discussed in the course of the previous lectures, type parametrization is a mechanism that allows us to factor out the common structure of different types into a single construction. That is true both of polymorphic functions and of parametric datatypes.

Queues, like any other container, represent the prototypical example of parametric datatype: a queue has the same shape regardless of what kind of element it holds — `Queue[Int]`, `Queue[String]`, `Queue[Point]` all share the same operations, differing only in the type of the elements.

To keep the focus on parametrization and variance — rather than on data structure design — we adopt a simple, list-based implementation of a functional queue:

```
class Queue[A] private (private val data: List[A]):
  def head: A                  = data.head
  def dequeue: Queue[A]        = new Queue(data.tail)
  def enqueue(a: A): Queue[A]  = new Queue(data ++ List(a))

object Queue:
  def apply[A](as: A*): Queue[A] = new Queue(as.toList)
```

The class declaration combines two ideas worth pointing out:

- The keyword `private` *before* the parameter list makes the *primary constructor* private. Clients cannot call `new Queue(...)` directly.
- The parameter `data` is itself declared `private val`, so it becomes a private immutable field of the class — invisible to clients.

Together with the public `apply` factory in the companion object, these declarations produce a proper abstract data type: clients construct queues only through `Queue(1, 2, 3)` (which expands to `Queue.apply(1, 2, 3)`, and from there to `new Queue(List(1, 2, 3))` — a call the companion object is permitted to make), and they manipulate queues only through the exposed methods `head`, `dequeue`, and `enqueue`.

The implementation is purely functional: `enqueue` and `dequeue` produce new queues without modifying any existing ones. We will see shortly that this fact alone does *not* settle the question of variance.

The rest of this lecture is about how subtyping interacts with type constructors: given that `A <: B`, when (and how) does it follow that `Queue[A] <: Queue[B]`?

Variance annotations

With its current definition, our parametric queue type does not support any subtype relationships among its possible instantiations. In fact, Scala parametrized types introduced with *unannotated* type variables, as our `Queue[A]`, are *invariant*: irrespective of the relationship that may hold between two types `S` and `T` instantiating the parameter `A`, `Queue[S]` and `Queue[T]` are simply *different* types.

In a concrete example, this means that `Queue[Shape]`, `Queue[Circle]`, and `Queue[Rectangle]` are all unrelated, even though `Circle <: Shape` and `Rectangle <: Shape`.

If we wish to make a parametrized type *variant*, this must be declared explicitly by tagging the type parameter with a *variance annotation* in its definition. Scala provides two such annotations:

- `Queue[+A]` — declares `Queue` as *covariant* in `A`: whenever `A <: B`, then `Queue[A] <: Queue[B]`. The subtype relation flows in the *same* direction as the parameter.
- `Queue[-A]` — declares `Queue` as *contravariant* in `A`: whenever `A <: B`, then Queue[B] <: Queue[A]`. The subtype relation flows in the *opposite* direction of the parameter.

The choice of invariance as the default is conservative — but, as we shall see, for good reason. Soundness places strong constraints on which variance annotations a type may carry, and not every type that one would *like* to make covariant can be.

Before we explore the annotations, let us first note that invariant containers as `Queue[A]` do not, per se, rule out useful uses of subtyping. To illustrate, consider the following snippet:

```
val mq: Queue[Shape] = Queue(Circle(1.0), Rectangle(3.5, 4.0))
```

This declaration typechecks even with our invariant `Queue[A]`. The reason is type inference:

- when the compiler sees the expected type `Queue[Shape]` on the left, it instantiates `Queue.apply[A]` with `A = Shape`
- now, the arguments `Circle(1.0)` and `Rectangle(3.5, 4.0)` are accepted as values of `Shape`, because `Circle <: Shape` and `Rectangle <: Shape`.

Thus the queue is constructed at the wider element type from the outset, and no subtype relation between `Queue[Circle]` and `Queue[Shape]` or `Queue[Rectangle]` and `Queue[Shape]` is ever needed.

Having said that, variant queues, in particular covariant queues, have interesting uses. Consider the following case, when a queue has already been constructed at a narrower element type, and we want to pass it to a function that expects a wider one:

```
def use(q: Queue[Shape]): Unit = ...

val cq: Queue[Circle] = Queue(Circle(1.0), Circle(2.0))

use(cq)               // needs Queue[Circle] <: Queue[Shape]
```

Without covariance, this call is rejected. The type of `cq` has already been fixed to `Queue[Circle]`, and the type checker has no way to relate `Queue[Circle]` and `Queue[Shape]`, even though `Circle <: Shape`. Type inference does not come to the rescue: the queue's element type is settled at its declaration (notice that this is true regardless of whether we annotate `cq`: the type inferred would anyway be `Queue[Circle]`), and only a variance relation can permit the upcast required at the call site.

Unfortunately, variancce annotation should be used with care, as they have subtle, harmful effects.

Covariant Queues require special care

In a purely functional settint, one might expect parametric types to be naturally covariant, as is the case for the depth subtyping rule for record types. As it turns out, however, this is subtler than it may appear.

To see this, assume we simply declare our queue covariant, by tagging the type parameter as `+A` in our implementation. That is, assume we change the definition as follows:

```
class Queue[+A] private (private val data: List[A]):
  def head: A                  = data.head
  def dequeue: Queue[A]        = new Queue(data.tail)
  def enqueue(a: A): Queue[A]  = new Queue(data ++ List(a))

object Queue:
  def apply[A](as: A*): Queue[A] = new Queue(as.toList)
```

Now, consider a subclass that specializes the queue for `Double`, overriding `enqueue` to do something that depends on the parameter being a `Double`:

```
class PowerQueue extends Queue[Double]:
  override def enqueue(a: Double): Queue[Double] =
    super.enqueue(math.pow(a, 2))        // squares before storing
```

The override is perfectly reasonable on its own: a `PowerQueue` is a `Queue[Double]` whose `enqueue` squares its argument before adding it to the queue. But under the covariance assumption, `Queue[Double] <: Queue[Any]`, so we can write:

```
val q: Queue[Any] = new PowerQueue       // by covariance, Queue[Double] <: Queue[Any]
q.enqueue("abc")                         // typechecks: "abc" is an Any
```

The call `q.enqueue("abc")` typechecks because the static type of `q` is `Queue[Any]`, whose `enqueue` accepts any `Any`. At *runtime*, however, dynamic dispatch sends the call to `PowerQueue.enqueue`, which expects a `Double` and applies `math.pow` to it. Applying `math.pow` to a string fails: the type system has been deceived.

The key observation is that the unsoundness depends on *inheritance and dynamic dispatch*, not on mutation. A subclass can always specialize the body of `enqueue` to assume something about the static parameter type — and covariant viewing breaks that assumption by allowing arguments of supertypes to flow in via dynamic dispatch.

There is nothing mysterious here, nor anything that should come as a surprise.

In fact, the problem is a direct consequence of the combined effect of record and function subtyping, as we have introduced them in our foundational presentation of subtyping.

To see that, we may see our `Queue` type by its public interface — that is, as a record type whose components are the operations the type exposes:

$$\mathsf{Queue}[\mathsf{A}] \;=\; \{\mathsf{head} : \mathsf{A},\ \mathsf{enqueue} : \mathsf{A} \to \mathsf{Queue}[\mathsf{A}],\ \mathsf{dequeue} : \mathsf{Queue}[\mathsf{A}]\}$$

By our record subtyping rule, given $\mathsf{A} <: \mathsf{B}$, we have $\mathsf{Queue}[\mathsf{A}] <: \mathsf{Queue}[\mathsf{B}]$ only in case all the component types vary covariantly, which requires that the following relations hold:

- $\mathsf{A} <: \mathsf{B}$,
- $\mathsf{A} \to \mathsf{Queue}[\mathsf{A}] <: \mathsf{B} \to \mathsf{Queue}[\mathsf{B}]$, and
- $\mathsf{Queue}[\mathsf{A}] <: \mathsf{Queue}[\mathsf{B}]$ (in the result of `dequeue`).

Now, while the first relation follows from our assumption $\mathsf{A} <: \mathsf{B}$, the second fails because the subtyping rule for function types requires $\mathsf{B} <: \mathsf{A}$ in the parameter position (and, instead, we have the opposite ordering!).

A covariant declaration `Queue[+A]` that enables the subtype relation `Queue[A] <: Queue[B]` when `A <: B` should therefore be rejected, as unsound. Indeed, attempting a covariant declaration of `Queue` produces the following reaction from the Scala type checker:

```
class Queue[+A] private (private val data: List[A]):
  def enqueue(a: A): Queue[A] = ...

-- Error: ----------------------------------------------------------------------
   |    def enqueue(a: A): Queue[A] = ...
   |                ^^^^
   |covariant type A occurs in contravariant position in type A of parameter a
```

Checking variance annotations

To verify the correctness of variance annotations, the Scala type checker classifies all *positions* in a class (or trait) as *positive*, *negative*, or *neutral*.

A *position* is any location in the type expressions of the class/trait body where a type parameter may be used:

- every method parameter type is a position,
- every method return type is a position,
- every field type is a position,
- argument and result of a function type is a position.

The classification follows two simple rules:

1. The body of the class is, by default, a positive position.
2. In a function type `S => T`, the sign of input position `S` is *flipped* relative to the surrounding context, while the output position `T` keeps the same sign as the surrounding context.

Once positions are classified:

- A type parameter declared `+A` may only appear in *positive* positions.
- A type parameter declared `-A` may only appear in *negative* positions.
- A type parameter with no annotation may appear *anywhere*.

The two rules compose: if a method has type `(A => B) => C` and we are looking at how `A` appears, we trace through two levels:

- the outer arrow has `(A => B)` in its input (negative) position;
- inside `(A => B)`, `A` is in the input (again negative) position, hence its sign *flips*, and is classified as *positive*

That is the same rationale as the variance behaviour of the input and output position in the subtyping rule for function types: output position as covariant, input positions are contravariant. But nesting a contravariant within a contravariant position makes it covariant: the negative of a negative is positive.

A worked example. Consider the methods of `Queue[A]` — but with `A` replaced by signs to track its variance:

```
class Queue[+A] private (private val data: List[A]):
  def head: A                                    // A in position +
  def dequeue: Queue[A]                          // A in position +
  def enqueue(a: A): Queue[A]                    // a:A in position -, return in position +
```

`head` returns an `A`: this is the body's positive position, so `A` appears positively. Same for `dequeue`, which returns a `Queue[A]`. `enqueue`, however, takes a parameter of type `A`: the parameter type is the input of a function, which flips the sign — so `a: A` puts `A` in a *negative* position.

If `A` is declared `+A`, that final case is a violation: the type checker reports exactly this, "covariant type A occurs in contravariant position." The annotation `+A` says we have promised to use `A` only in positive positions; the method `enqueue` breaks that promise.

The technical mechanisms used by the type checker are subtler than this in some details, but the intuition above captures the essence.

Recovering a sound `Queue[+A]`

The trick to recover a sound implementation of covariant queues is to generalize the definition of `enqueue` by making it polymorphic in any supertype of the type of the queue elements:

```
class Queue[+A] private (private val data: List[A]):
  def head: A           = data.head
  def dequeue: Queue[A] = new Queue(data.tail)
  def enqueue[U >: A](a: U): Queue[U] = new Queue(data ++ List(a))

object Queue:
  def apply[A](as: A*): Queue[A] = new Queue(as.toList)
```

Unlike the original definition of `enqueue`, which requires the types of the new elements to have *exactly* the type `A` introduced in the `Queue[+A]` declaration, the new version accepts arguments of any `U` supertype of `A` (and, correspondingly, returns a `Queue[U]` instead of `Queue[A]`).

The revised version of `enqueue` is type correct: technically, this is because the the lower bound `U >: A` introduces a negative position, and `A` occurs negatively in that position. Intuitively, the lower bound `U >:A` anticipates any possible use of subtyping on the container `Queue[A]` by adjusting the assumption on the type of the enqueued element accordingly. Therefore, the new version of `enqueue` is also sound

Back to our foundational presentation, this can be explained as follows. The new `Queue` type can be seen as a record type of the following form:

$$\mathsf{Queue}[A] \;=\; \{\ldots, \mathsf{enqueue} : [U >: A](U \to \mathsf{Queue}[U]), \ldots\}$$

By record subtyping , given $A <: B$:

- $\mathsf{Queue}[A] <: \mathsf{Queue}[B]$ only if $[U >: A](U \to \mathsf{Queue}[U]) \;<:\; [U >: B](U \to \mathsf{Queue}[U])$
- which in turn requires $[U >: B]U \;<:\; [U >: A]U$
- which holds, as the supertypes of B are supertypes of A when $A <: B$

Indeed, by assuming that the type of the `enqueue` parameter is *any* supertype `A`, we leave the type checker with *no information* on how the parameter can be operated upon (except for being passed around). In particular, a subclass cannot override `enqueue` to specialize on a particular static parameter type — and so the unsoundness witness we exhibited earlier no longer typechecks:

```
class PowerQueue extends Queue[Double]:
  override def enqueue[U >: Double](a: U): Queue[U] =
    super.enqueue(math.pow(a, 2))

-- Type Mismatch Error: ------------------------------------------------
   |    override def enqueue[U >: Double](a: U): Queue[U] =
   |                                               super.enqueue(math.pow(a,2))
   |                                                                        ^
   |           Found:    (a : U)
   |           Required: Double
   |           where:    U is a type in method enqueue with bounds U >: Double
```

The override must accept any `U >: Double`, but its body tries to pass `a` to `math.pow`, which expects a `Double`. The type checker rejects the override at compile time, because there is no guarantee that the runtime argument is a `Double` — it may be any supertype of it. The `PowerQueue` specialization simply cannot exist in the variant world.

When invariance unavoidable

The lower-bound trick works because enqueue is purely functional: it returns a new queue, whose type can be widened along with the parameter. The widening propagates outward through the return type, and no existing queue is affected.

When the queue is mutable, the trick breaks down. Consider:

```
import scala.collection.mutable.ArrayBuffer

class MutableQueue[A]:
  private val buf = ArrayBuffer.empty[A]
  def enqueue(x: A): Unit = buf += x
  def head: A             = buf(0)
  def dequeue(): A        = buf.remove(0)
```

Suppose we tried to make this covariant by widening the enqueue parameter:

```
class MutableQueue[+A]:
  private val buf = ArrayBuffer.empty[A]
  def enqueue[U >: A](x: U): Unit = buf += x   // hopeful, but wrong: the assignment would requ
  def head: A                     = buf(0)
  def dequeue(): A                = buf.remove(0)
```

The bounded enqueue widens its parameter to any `U >: A`, just as in the functional case. But now there is no return type to carry the widening: enqueue updates the queue in place, which is where the contruction falls apart, because the update would require `U <: A`, while the bound requires `U :> A`. In the end, `U = A` and we are back to the previous implementation, where the position calculus finds both positive and negative occurrences of `A` and hence force `MutableQueue` to be invariant in `A`.

The conclusion: in mutable parametric mutable parametric containers, which expose both update (*consume*) and access (or *produce*) operations on their content must remain invariant.

Contravariant types and upper bounds

Not surprisingly, there exist datatypes that have a natural contravariant nature. In fact, whereas covariant types are inherently associated with containers that *produce* values, contravariant types arise naturally for datatypes that can be construed as *consumers* of values. Consider, for instance, the case of output channels defined below:

```
trait OutputChannel[-A]:
          def write(x: A) : Unit
```

Though it might at first seem counter-intuitive, the contravariant definition of output channel makes perfect sense. To see that, consider that the only supported operation on a channel of type, say `OutputChannel[String]` is to write a `String` to it. But the same operation can also be done on an `OutputChannel[AnyRef]` (as long as we don't read back from the channel) so it is safe to substitute `OutputChannel[AnyRef]` for `OutputChannel[String]`. As soon as we "read" from the channel, the contravariant annotation is rejected:

```
trait OutputChannel[-A]:
          def write(x: A) : Unit
          def last : A
-- Error: ----------------------------------------------------------------------
   |    def last : A
   |    ^^^^^^^^^^^^
   |contravariant type A occurs in covariant position in type => A of method last
```

The problem is the exact dual of the corresponding problem with covariant definitions, and the solution can be obtained by applying the same duality principle:

```
trait OutputChannel[-A]:
          def write(x: A) : Unit
          def last[U <:A] :  U
```

Supplementary Material — Scala variant types vs Java's wildcards

Variance annotations provide a convenient alternative to traditional mechanisms available in object-oriented languages to enable flexible uses of data structures based on subtyping. One relevant example is Java, with its system of wildcards and (bounded) polymorphic types. We look at the tradeoffs between Java's wildcards and Scala's variant types below.

As we will show, variant types appear superior:

- when designing class code:
  - the subtyping flexibility of variant traits or classes can be achieved with invariant traits and classes,
  - but the price to pay is additional complexity in the design of methods within traits /classes;
- when designing client code:
  - invariant class definitions require effort for clients by means of use-site variance mechanisms.

In both cases, lower bounds and upper bounds play a key role.

We illustrate the point with the design and use of (immutable) `Streams` , and the `filter` and `map` methods which have a related structure in Java and Scala (streams are deprecated in Scala, but the exact same arguments apply to LazyLists). In Scala, we have the following, rather direct definition:

```
trait Stream[+A]:
          . . .
          def filter(p: A => Boolean) : Stream[A]
          def map[B](f: A =>  B): Stream[B]
```

The Java version of the same type is more contrived.

```
interface Stream<T>
    . . .
         Stream<T> filter(Predicate<? super T> p)
         <R> Stream<R> map(Function<? super T,? extends R> f)`
```

To ease the discussion, it is useful to start with reconciling the syntactic conventions in the two languages, by

- "decoding" the types `Predicate<T>` and `Function<T,U>` and exposing their true nature as the function types `T => Boolean` and `T => U`, respectively, and
- expressing the `? super T` and `? extend T` notation in terms of its Scala upper and lower bound correspondents.

Then, we may rewrite the Java definition in Scala syntax as follows:

```
trait JavaStream[T]
    . . .
         def filter[U >:T](p : U => Boolean): JavaStream[T]
         def map[R, U >: T, S <: R] (f : U => S): JavaStream[R] `
```

You may wonder whether the additional structure of lower and upper bounds used in the invariant `JavaStream` trait is really needed: intuitively, that structure allows the two methods to be used at supertypes of the type `T` of the enclosing `JavaStream` so as to recover the flexibility of the variant `Stream[+A]` trait.

To illustrate, we define a function that, given a stream of points and a line, selects the points that belong to the line. The following definitions, using the variant `Stream` trait, serves the purpose:

```
class Line(val m: Double, val q: Double)

def belongs(l : Line)(p: Point): Boolean = (p.y == l.m * p.x + l.q)
def pointsOnLine(pts : Stream[Point], l: Line) = pts.filter(belongs(l))
```

Now assume we wish to reuse the previous definition on a stream of colored points.

With the variant `Stream` trait the reuse is immediate, as `Stream[ColorPoint] <: Stream[Point]` hence a `Stream[ColorPoint]` can be simply passed to `pointsOnLine`.

```
val coloredPoints : Stream[ColorPoint] = . . .
pointsOnLine(coloredPoints, new Line(2,5))
```

With the invariant `JavaStream` instead, we must necessarily refine the definition of our `pointsOnLine` function, so as to make it reusable with streams of subtypes of `Point`. Lower bounds come to the rescue, here, as we can rewrite our function as a polymorphic function with a lower bound:

```
def javaPointsOnLine[U <: Point](pts : JavaStream[U], l: Line) =
          pts.filter(belongs(l))

val javaColoredPoints : JavaStream[ColorPoint] = . . .
javaPointsOnLine(javaColoredPoints, new Line(3,5))
```

The type checker accepts `javaPointsOnLine(javaColoredPoints)` by instantiating `U` to `ColoredPoint`, which is correct as `ColoredPoint <: Point`: that is how the combined use of a polymorphic stream and subtyping of the component type captures the effect of subtyping over the stream type.

It is instructive to look at the invocation `pts.filter(belongs(l))` and to detail its typing, as it this is where the polymorphic type bound used in the definition of `JavaStream.filter` comes into the picture. In fact.

- on the one hand `pts : JavaStream[U]` with `U <: Point`
- on the other `filter` accepts predicates whose input type is a supertype of `U`:
- but that's exactly the case for `belongs(l): Point => Boolean`, given that `U <: Point`

Viewed differently, `belongs(l) : Point => Boolean` may legally be passed for the the parameter `p: U => Boolean`, because `Point => Boolean <: U => Boolean` given that `U <: Point` (and the fact that function types are contravariant in their input).

References and Credits

The first part of this lecture is based on Chapter 18 of *Programming in Scala*.

The comparison between Scala and Java is inspired by (but substantially different from) S. Crafa's lecture note and slides from her course on Advanced Topics in Programming Languages

Upper bounds are an application of the principle of *bounded quantification* introduced by L. Cardelli and P. Wegner in their renown paper on Understanding Types, Data Abstraction and Polymorphism.